Modul 4.2: Prozesse steuern

Anschluss an Modul 4.1

Im letzten Modul hast du Prozesse mit `ps`, `top` und `htop` beobachtet. Jetzt steuerst du Prozesse gezielt: Du unterbrichst Vordergrundbefehle, beendest Prozesse mit Signalen und verwaltest pausierte und hintergrundlaufende Jobs.

> **Caution**
>
> Steuere ausschließlich die vorbereiteten Prozesse `monitoring.sh`, `sturkopf.sh` und deine eigenen Testjobs. Prüfe die PID unmittelbar vor dem Signal und verwende `SIGKILL` (`kill -9`) nur als letzte Eskalation nach erfolglosem `SIGTERM`.

1. Der hängende Prozess

Sieh dir die laufenden Prozesse an:

```
top
```

Suche `monitoring.sh` und `sturkopf.sh` in der Liste. Beende `top` mit `q`. Merke dir nur die Namen; die PIDs ändern sich und müssen immer frisch gesucht werden.

2. Strg+C: einen Befehl abbrechen

Starte einen Vordergrundprozess:

```
sleep 60
```

Drücke `Strg+C`. Das Terminal zeigt `^C` und kehrt zur Eingabeaufforderung zurück. `Strg+C` sendet `SIGINT` (Signal 2). Viele Programme beenden sich daraufhin kontrolliert.

3. kill: einen Prozess gezielt beenden

Finde den Monitoring-Prozess:

```
ps aux | grep monitoring.sh
```

Nimm die PID der Zeile mit `/usr/local/bin/monitoring.sh`, nicht die PID des `grep`-Befehls. Beende ihn ordentlich:

```
kill <PID>
```

`kill` sendet standardmäßig `SIGTERM` (Signal 15). Der Prozess erhält die Chance, Dateien zu schließen und aufzuräumen.

Prüfe den Status:

```
ps -eo stat,cmd | grep monitoring.sh | grep -v grep
```

`S`, `R` oder ähnliche normale Statuswerte bedeuten: Der Prozess läuft noch. `Z` oder `<defunct>` bedeutet in diesem Labor: Der Prozess läuft nicht mehr, sondern wartet nur noch auf die Aufräumarbeit seines Elternprozesses.

4. kill -9: wenn ein Prozess nicht reagiert

Finde den widerspenstigen Prozess:

```
ps aux | grep sturkopf.sh
```

Versuche zuerst das normale Signal:

```
kill <PID>
sleep 3
ps aux | grep sturkopf.sh
```

`sturkopf.sh` ignoriert `SIGTERM` absichtlich. Sende deshalb erst jetzt `SIGKILL`:

```
kill -9 <PID>
```

Prüfe:

```
ps -eo stat,cmd | grep sturkopf.sh | grep -v grep
```

`SIGKILL` (Signal 9) wird direkt vom Kernel durchgesetzt. Der Prozess kann nicht aufräumen. Deshalb gilt die Faustregel: erst `kill`, nur bei Bedarf `kill -9`.

Signal	Befehl	Wirkung
`SIGINT` (2)	`Strg+C`	Vordergrundbefehl unterbrechen
`SIGTERM` (15)	`kill PID`	kontrolliertes Beenden anfordern
`SIGKILL` (9)	`kill -9 PID`	sofort beenden, nicht ignorierbar

5. SIGHUP

Starte einen eigenen Testprozess:

```
sleep 300 &
```

Sende ihm `SIGHUP` (Signal 1). Verwende die PID aus der Ausgabe:

```
kill -1 <PID>
jobs
```

`SIGHUP` bedeutet ursprünglich „Hangup“, also eine getrennte Terminalverbindung. Viele Prozesse beenden sich, wenn ihr Terminal oder ihre SSH-Verbindung verschwindet. Manche Dienste verwenden `SIGHUP` auch, um ihre Konfiguration neu zu laden.

6. Strg+Z und fg: Prozesse pausieren

Starte:

```
sleep 120
```

Drücke `Strg+Z`. Der Job ist jetzt pausiert und erhält eine Jobnummer, zum Beispiel `[1]`. `Strg+Z` sendet `SIGTSTP` (Signal 20); es beendet den Prozess nicht.

Hole ihn zurück in den Vordergrund:

```
fg
```

Beende ihn anschließend mit `Strg+C`.

Tastenkombination	Signal	Wirkung
`Strg+C`	`SIGINT`	Prozess beenden oder unterbrechen
`Strg+Z`	`SIGTSTP`	Prozess pausieren

7. & und jobs

Starte zwei eigene Hintergrundjobs:

```
sleep 300 &
sleep 600 &
```

Zeige die Jobs deiner aktuellen Shell:

```
jobs
```

`jobs` zeigt nur Prozesse, die von dieser Shell gestartet wurden. Für alle Prozesse auf dem Server verwendest du weiterhin `ps aux`.

Hole den zuletzt gestarteten Job in den Vordergrund:

```
fg
```

Beende ihn mit `Strg+C`. Beende anschließend auch den anderen Testjob kontrolliert, falls er noch läuft:

```
jobs
```

Nutze die angezeigte Jobnummer mit `fg %1` oder `fg %2` und anschließend `Strg+C`.

8. Der Monitoring-Prozess hängt wieder

Die Laborumgebung startet `monitoring.sh` nach kurzer Zeit erneut. Falls er noch nicht wieder sichtbar ist, warte einige Sekunden und prüfe:

```
ps aux | grep monitoring.sh
```

Finde seine PID und beende ihn mit dem normalen Signal:

```
kill <PID>
```

Starte danach einen eigenen Testprozess im Hintergrund:

```
sleep 500 &
```

Zeige deine Jobs:

```
jobs
```

Beende den Testjob am Ende mit `fg` und `Strg+C`, damit keine unnötigen Prozesse zurückbleiben.

Sicherheitsregeln

- Nur die vorbereiteten Trainingsprozesse und eigene `sleep`-Jobs steuern.
- PIDs immer unmittelbar vor dem Signal neu ermitteln.
- Erst `SIGTERM`, nur bei nachweislicher Reaktionlosigkeit `SIGKILL`.
- Niemals `kill -9` pauschal oder gegen unbekannte Systemprozesse verwenden.
- Keine Prozesse anderer Nutzer außerhalb dieses Labs beenden.
- Zurückbleibende Testjobs vor dem Ende mit `jobs` prüfen und beenden.

Zusammenfassung

Du kannst jetzt:

- Vordergrundprozesse mit `Strg+C` unterbrechen,
- Hintergrundprozesse mit `kill` und einer PID beenden,
- `SIGTERM` und `SIGKILL` unterscheiden,
- `SIGHUP` praktisch testen,
- Prozesse mit `Strg+Z` pausieren und mit `fg` fortsetzen,
- eigene Hintergrundjobs mit `&` und `jobs` verwalten.

Im nächsten Modul untersuchst du Prozesse gezielter nach Namen, PID und Beziehungen.